

AI-Native Hardware Validation

Building a Local-Context Agentic Test and Debug Platform

Wenxi Yan *Principal Systems Engineer, Microsoft*

Cheng Ying Chen *Principal Systems Engineer, Microsoft*

OCP PLATINUM Member

Problem Statement: Will We Let the AI Own the Loop?

The model is powerful, but the lab context is missing.
Legacy validation gave AI only isolated tasks: write a script, explain a log, summarize a bug.

AI-native validation asks a harder question:

What must we expose so an AI agent can safely run, observe, reason, and update the test loop?



Core thesis: context + tools + guardrails turn a generic assistant into a hardware-validation agent.

Three OCP Talks, From Assisted Automation to Agent-Owned Validation

Year 2024

OCP Talk #1
copy-paste: AI writes scripts/debug

Year 2025

OCP Talk #2
AI **triages** logs with pre-defined error signatures

Year 2026

OCP Talk #3
AI **execute**, drives the full **loop**

AI now proven in practice

1. Connect nodes and run tests
2. Collect, decode, and fingerprint logs
3. Search bug history and reproduce on target nodes
4. Reason with specs, KB, and coverage
5. Write back bugs, test status, and documents

My 3-year OCP story mirrors the field's jump: AI-assist → AI-led validation.

AI-Native Validation Stack: Where the Differentiation Lives

Fast-moving external layers — use best available

Foundation Models

Codex / Claude class core

Powerful, evolving fast

Agent Harness

CLI agent
persistent sessions
ACP/SDK/API calls

We adopt, not create derivatives.

Our value-add — MCPs, skills and GUI

Context

KB
- Bug system
- Specs
- Test cases system

Execution

McConsole
- Unique commands
- Seamless GUI
- Memory as needed

Log_Analyzer
- Unique Error signatures

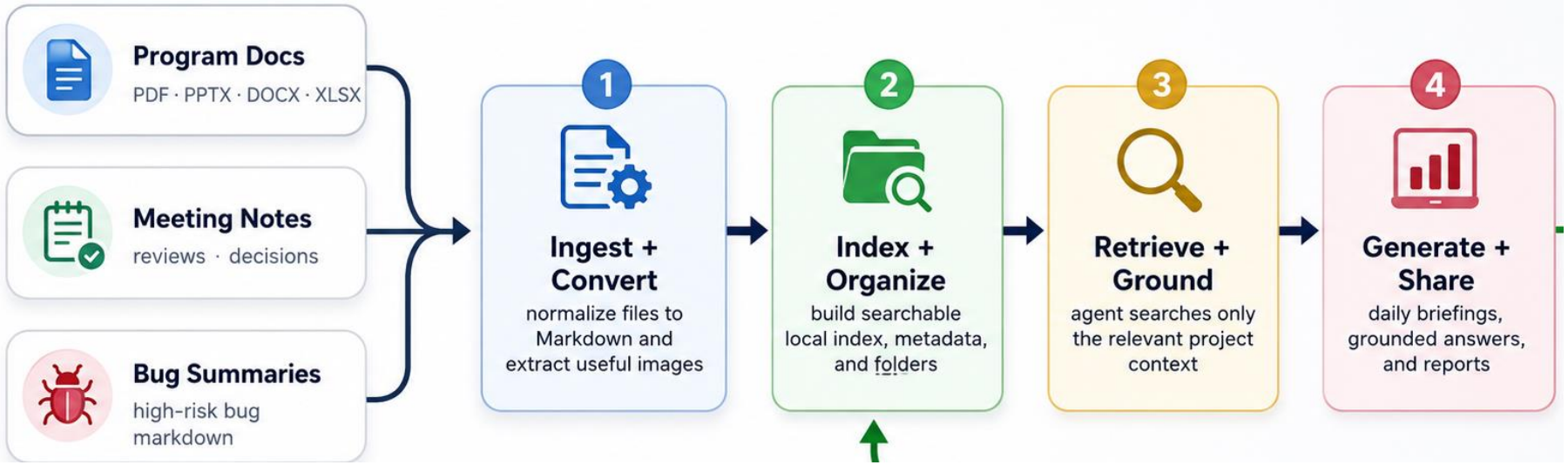
Records

Ocean of Logs, telemetry, artifacts
- file-system-first store
- every byte captured
- Write back to Bug/spec/test case systems as needed

We scale the lab by making the unique tools and data usable to the agent.

Value add: Context— Technical Project Knowledge

Knowledge Base — scheduled refresh pipeline



What it adds

- Scheduled jobs periodically dump project docs, meeting notes, and bug summaries
- Files are converted to Markdown and indexed for fast, project-scoped retrieval
- The agent searches only the relevant local knowledge during test and debug

Value add: Execution— MCP/Skill-Driven Command Execution

Engineer

intent
target servers
requested action

Typical requests

- Find X # of servers and try to reproduce bug Y
- Connect with Server_X, Console_Y, to run/check test Z
- Update Server X FW to recipe X
- Do 1-for-all config check, and logs dump with Server X.
- Use available machine pool, find proper IO test cases to run in this week.

Controlled execution layer

Command catalog

platform × console types
reusable command patterns

Execution router

target resolution
console selection
session setup

Console adapters

standard transports
specialized adapters

Target system + evidence

SUT control surfaces

host OS
management controller
board / fabric / recovery console

Tests + diagnostics

run workload / cycle / health check

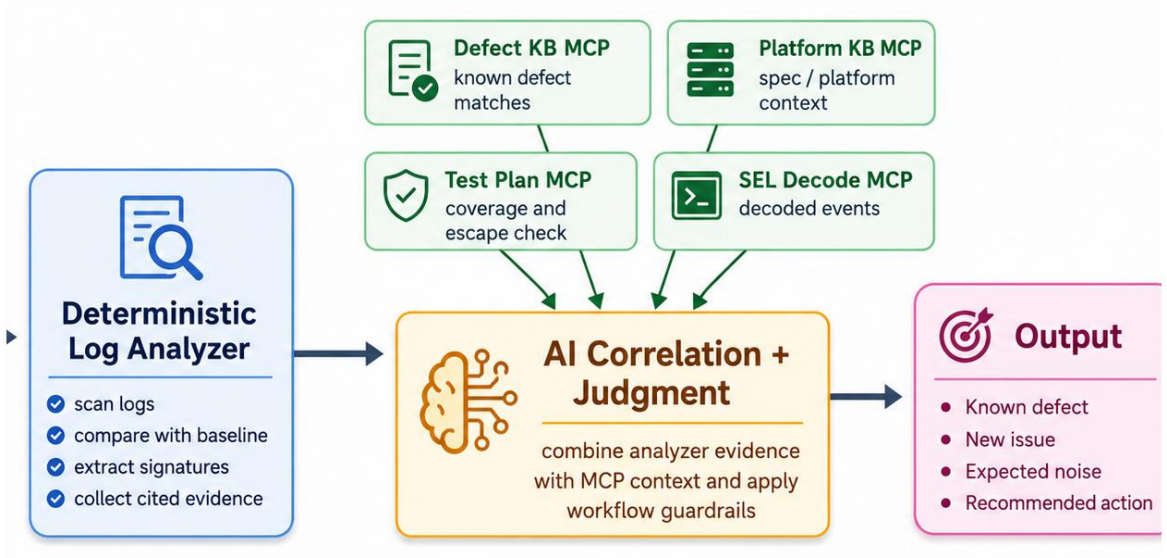
Logs + telemetry

collect raw evidence
OCP T&V JSONL (proposed)

Execution contract: intent → command template → execution router → adapter → SUT → log archive.

Value add: Execution— Autonomous Log Analysis

Autonomous Log Analyzer — evidence + MCP context



What it adds

- Raw-log signatures today; proposed OCP T&V JSONL adapter for step/diagnosis analysis
- MCP and skills bring in defect KB, platform KB, test-plan, and SEL decode context
- AI correlates the evidence and returns known defect, new issue, or expected noise

Value add: Records — Turn Log & Telemetry Archives into Searchable Evidence

Log archive

Lab log root

Node / SUT folders
all runs from one system

Test-run folders
date · type · version

Raw logs
SEL · sensors · OS · test output

Analyzer outputs
signatures · summaries · reports

Scheduled indexer
scan + extract + aggregate

OCP JSONL archive (planned)

Indexed discovery

Compact log index

run metadata
OCP step IDs (planned)
error signatures
recurrence catalog

Log search layer

read-only MCP or scaled service
FTS5 / SQL queries
node / date / signature lookups

Answers "where should AI look?"

Engineer

Review auto generated report
Intent- for specific ask

Typical questions

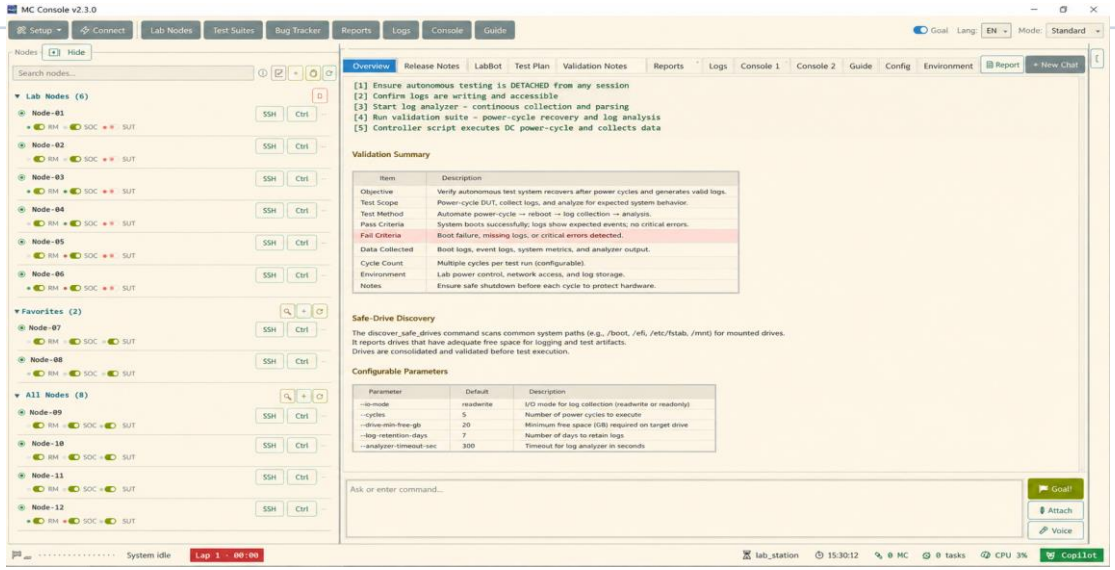
- Was signature X seen last week?
- Which systems repeat this fault?
- How did weekend testing go?
- What exactly happened in this failed run?

Raise request or review auto reports

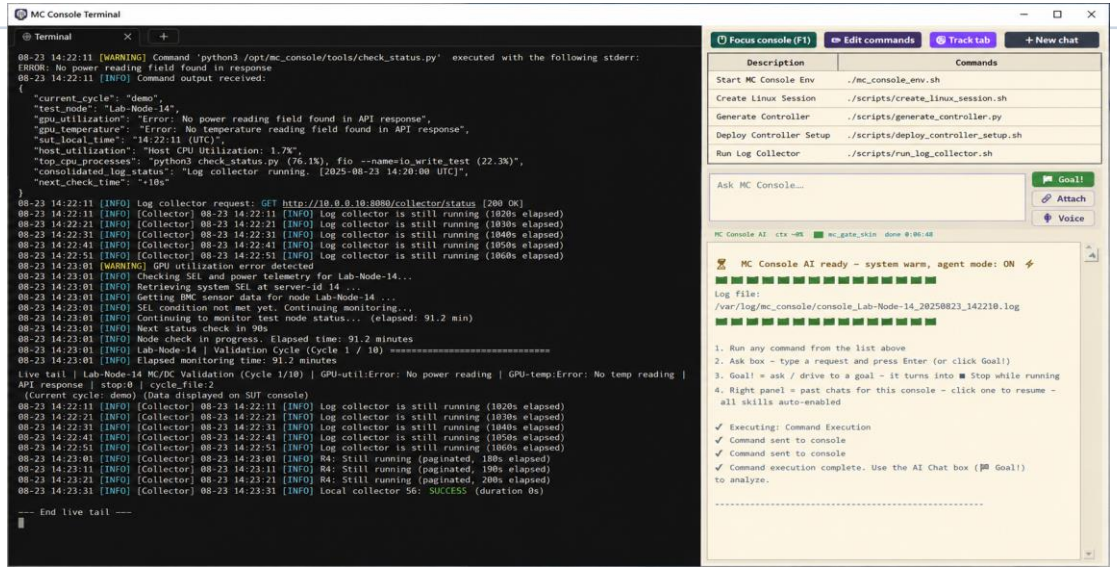
Value add— McConsole GUI for seamless front-end

1	2	3	4	5
Connect	**Run**	**Observe**	**Reason**	**Update**
Acquire node, console, OOB, in-band sessions	Launch test case or reproduce known failure	Capture logs, telemetry, events, artifacts	Decode signatures, search bugs, inspect KB	Bug, test plan, coverage, handoff notes

Lab view, multi AI chats



Live execution in terminals with AI cmds sidecar



Call to Action — Share, reuse, and contribute practical building blocks for AI-native validation.

1 **Use every lab node**

Expose available systems, consoles, test goals, and repro requests through agent-friendly workflows.

2 **Autonomous execution**

Let agents plan weekend campaigns: I/O, GPU high-power, bug repro, health checks, and log collection.

3 **Search every signal**

Index logs, telemetry, decoded events, bug history, specs, and test plans so AI can find the evidence quickly.

4 **Learn every run**

Write evidence, verdicts, escapes, and status back into bugs, knowledge bases, and coverage systems.

github.com/microsoft/log_analyzer • github.com/microsoft/mcconsole
github.com/opencomputeproject/ocp-diag-core • github.com/opencomputeproject/ocp-diag-core-python

Thank You!

